

UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Name: Operating System Lab

Course Code: CSE 402

LAB REPORT

Lab Report Number: 07

Lab Report Name: Implementation and Comparison of FCFS and SSTF Disk Scheduling Algorithms

Submitted By:	**Submitted To:**
Name: Tashnimul Hasan Shifat	Atia Rahman Orthi
Reg: 23101142	Lecturer
Section: C2	University of Asia Pacific

Date of Submission: 04.10.2026

1. Problem Statement:

In a computer system, multiple processes frequently issue I/O requests to the disk. Since the disk has a single read/write head that must physically move between cylinders, the seek time is the dominant component of disk access time. The operating system therefore uses disk scheduling algorithms to decide the order in which pending requests are serviced, with the aim of minimizing total head movement.

The objective of this laboratory task is to implement the First Come First Serve (FCFS) and Shortest Seek Time First (SSTF) disk scheduling algorithms. FCFS services the requests strictly in the order in which they arrive, whereas SSTF always selects the pending request that requires the minimum head movement from the current head position. Given a request queue of 3, 60, 10, 25, 30, 4 and an initial head position of 41, the program must determine the seek sequence and the total head movement for each algorithm and compare the results to identify the more efficient algorithm.

2. Objectives:

- To understand the working principles of the FCFS and SSTF disk scheduling algorithms.
- To implement both algorithms in Python using a modular, function-based design.
- To determine the seek sequence and total head movement for each algorithm.
- To compare the performance of both algorithms based on total head movement.
- To analyze the advantages, limitations and time complexity of each algorithm.

3. Source Code Screenshot:

```python

def fcfs(requests, head):
    """Serve requests in the order they arrive."""
    order = [head]
    movement = 0
    current = head

    for r in requests:
        movement += abs(current - r)
        current = r
        order.append(r)

    return order, movement


def sstf(requests, head):
    """Always serve the pending request closest to the current head."""
    order = [head]
    movement = 0
    current = head
    pending = requests.copy()

    while pending:
        nearest = min(pending, key=lambda r: abs(current - r))
        pending.remove(nearest)

        movement += abs(current - nearest)
        current = nearest
        order.append(nearest)

    return order, movement


```

Figure 1: Implementation of the fcfs() and sstf() functions

```
33
34  def show_result(name, order, movement):
35      print(f"\n{'=' * 10} {name} {'=' * 10}")
36      print("Seek Order:", " -> ".join(map(str, order)))
37      print("Total Head Movement =", movement)
38
39
40  def compare(fcfs_move, sstf_move):
41      print(f"\n{'=' * 10} COMPARISON {'=' * 10}")
42      print("FCFS Total Head Movement =", fcfs_move)
43      print("SSTF Total Head Movement =", sstf_move)
44
45      diff = abs(fcfs_move - sstf_move)
46
47      if sstf_move < fcfs_move:
48          print("\nSSTF is better than FCFS.")
49          print(f"SSTF has less head movement by {diff} cylinders.")
50      elif fcfs_move < sstf_move:
51          print("\nFCFS is better than SSTF.")
52          print(f"FCFS has less head movement by {diff} cylinders.")
53      else:
54          print("\nBoth SSTF and FCFS have the same head movement.")
55
56
```

Figure 2: Implementation of the show_result() and compare() functions

```
57  def main():
58      requests = [3, 60, 10, 25, 30, 4]
59      head = 41
60
61      sstf_order, sstf_move = sstf(requests, head)
62      fcfs_order, fcfs_move = fcfs(requests, head)
63
64      show_result("SSTF", sstf_order, sstf_move)
65      show_result("FCFS", fcfs_order, fcfs_move)
66      compare(fcfs_move, sstf_move)
67
68
69  if __name__ == "__main__":
70      main()
```

Figure 3: Implementation of the main () function

4. Output:

```
========== SSTF ==========
Seek Order: 41 -> 30 -> 25 -> 10 -> 4 -> 3 -> 60
Total Head Movement = 95

========== FCFS ==========
Seek Order: 41 -> 3 -> 60 -> 10 -> 25 -> 30 -> 4
Total Head Movement = 191

========== COMPARISON ==========
FCFS Total Head Movement = 191
SSTF Total Head Movement = 95

SSTF is better than FCFS.
SSTF has less head movement by 96 cylinders.

[Done] exited with code=0 in 0.166 seconds
```

Figure 4: Execution output of the FCFS and SSTF disk scheduling program

5. Discussion:

The implemented First Come First Serve (FCFS) and Shortest Seek Time First (SSTF) disk scheduling algorithms represent two distinct approaches to servicing disk I/O requests. FCFS processes the requests strictly in the order in which they are received, whereas SSTF selects, at each step, the pending request that requires the minimum head movement from the current position of the disk head.

The program accepts the disk request sequence and the initial head position as input and generates the sequence for both algorithms. The total head movement is then computed as the sum of the absolute distances travelled by the disk head between consecutive requests. The following observations were made from the experiment:

- In FCFS, the requests are serviced sequentially in their given order. The resulting seek sequence is 41 → 3 → 60 → 10 → 25 → 30 → 4, which yields a Total Head Movement of 191 cylinders.
- In SSTF, the pending request closest to the current head position is selected at each step. The resulting seek sequence is 41 → 30 → 25 → 10 → 4 → 3 → 60, which yields a Total Head Movement of 95 cylinders.
- SSTF outperforms FCFS for the given dataset, reducing the total head movement by 96 cylinders, which corresponds to an improvement of approximately 50.3%.
- FCFS is straightforward to implement, as it does not require any search for the nearest request. Its request-processing operation therefore has a time complexity of O(n).

- The implementation of SSTF repeatedly scans the remaining requests to identify the closest one, resulting in a time complexity of $O(n^2)$. This additional computational cost is, however, offset by a considerably lower total head movement.
- FCFS is fair and predictable, since every request is serviced according to its order of arrival. Nevertheless, it may lead to excessive head movement when consecutive requests are located far apart on the disk.
- SSTF minimizes unnecessary head movement by servicing nearby requests first; however, requests located far from the current head position may be delayed indefinitely, leading to starvation.

In summary, the experiment provided practical insight into disk scheduling, seek sequence generation and the calculation of total head movement. Based on the experimental results, SSTF is more efficient than FCFS for the given dataset, achieving a total head movement of 95 cylinders compared to 191 cylinders for FCFS.

6. Conclusion:

In this experiment, the FCFS and SSTF disk scheduling algorithms were successfully implemented in Python and evaluated using the request queue 3, 60, 10, 25, 30, 4 with an initial head position of 41. FCFS produced a total head movement of 191 cylinders, whereas SSTF produced 95 cylinders, a reduction of 96 cylinders. Therefore, SSTF performed considerably better than FCFS for the given input.

It can be concluded that FCFS is suitable where simplicity and fairness are the primary concerns, while SSTF is preferable when minimizing seek time is the main objective, provided that the possibility of starvation is acceptable or handled appropriately. The experiment also provided practical understanding of seek sequence generation, head movement calculation, and the advantages and limitations of both algorithms.